Canonical source: 00-CHARTER/GUIDES/ATOMIC_ORANGE_NATIVE_APP.md

Æ Orange AI Computer Atomic Orange Native App

Atomic Orange is the native operator surface for Æ Orange AI Computer. It is a Tauri application, not the owner of intelligence, memory, policy, execution, or receipts. Closing or replacing the interface must not erase the governed system behind it.

Operator Contract

The app should make one work identity legible from request to result:

```
operator request
-> native bridge
-> OrangeLLM or Brain MCP crossing
-> route and model identity
-> streamed visible work
-> BuildRun state
-> report and receipt identity
-> operator-visible completion or blocker
```

The interface may explain and present. It may not manufacture a receipt, hide a failed gate, turn a configured feature green, or relabel a model route after the fact.

Expected Surfaces

- conversation with real incremental streaming and Stop/cancel;
- active model, route, host, and health state;
- BuildRuns with active, completed, blocked, cancelled, and failed states;
- receipt and evidence access for completed work;
- memory and project context without exposing secrets;
- bounded tool or agent approvals;
- visual and creative artifacts linked to provenance;
- recovery information that names the first failed gate.

Controls should remain useful when the heavy model host is unavailable. The N150 control plane can continue deterministic health, receipt, and project work; Codexa supplies heavy inference and long jobs when reachable.

Native Proof Standard

A browser preview is useful for frontend iteration but is not native proof. Acceptance requires the packaged Tauri executable and records:

- native process identity and executable path;
- gateway or model request identity;
- time to first visible assistant content;
- final completion or explicit cancellation;
- BuildRun identity and chain state;
- receipt identity visible in the app;
- screenshot hash and native runtime evidence.

The accepted Blue Bench includes one native conversation proof. A later build must earn a new receipt before it replaces that result. Backend success alone does not prove the bridge, streaming, cancellation, rendering, or visible state.

Failure Boundaries

Symptom	First check
App opens but chat stalls	semantic `/healthz`, route, and first upstream content
Text arrives only at the end	native bridge buffering and stream forwarding
Stop changes UI only	cancellation propagation to native request and upstream generation
Run disappears	BuildRun persistence and projection, never ledger deletion
Receipt is shown without result	shared run identity and chain validation
Visual looks correct but cannot be audited	source hash, route, report, and receipt linkage

---

Safe Operation

Do not solve a native problem by widening a loopback service to the public network, killing unrelated Bun or model processes, deleting ledgers, or moving authority into UI state. Identify ownership, preserve evidence, repair the first causal failure, then rerun the native crossing.

Related Guides

- Quick Start
- Operator Manual
- Receipts and Audit
- Troubleshooting and Recovery
- Proof and Benchmarks